FLEXI–Haz: A Python Implementation of Partially Linear and Fully Nonparametric Neural Hazard Models

Matan Bar Tov

Research Exposure Program 2026

1 Introduction

Survival analysis concerns the modeling of time-to-event outcomes in the presence of censoring and is widely used in areas such as medicine, epidemiology, and reliability analysis. A central challenge is to model potentially complex relationships between covariates and survival while retaining interpretable effects for variables of primary scientific interest. The classical Cox proportional hazards model of Cox (1972) provides a convenient framework for interpretation, but relies on the proportional hazards assumption and typically models covariate effects through a relatively restrictive linear predictor. Neural-network survival models can provide substantially greater flexibility, although many existing approaches remain based on the proportional hazards structure, which limits their ability to represent time-varying covariate effects.

FLEXI–Haz, introduced by Ben Arie and Gorfine (2025), addresses this limitation through a partially linear neural hazard model. Rather than specifying a separate baseline hazard, the conditional log-hazard is modeled directly as

$$\log h(t \mid X, Z) = \theta^\top Z + g(t, X),$$

where the finite-dimensional parameter θ provides interpretable effects for selected covariates and the neural-network component $g(t, X)$ captures nonlinear covariate effects, time dependence, and time-covariate interactions. This formulation therefore relaxes the global proportional hazards assumption while retaining statistical inference for the covariates represented through Z. The original work, however, primarily focused on the statistical methodology and its theoretical properties, leaving substantial room for a more complete and practical software implementation.

The main goal of this project was therefore to develop FLEXI–Haz into a practical, efficient, and extensible Python package for neural survival analysis. A central design decision was to separate the model into two user-facing specifications. The first implements the original partially linear formulation, in which selected covariates are represented through the interpretable coefficient θ. The second is a new fully nonparametric variant,

$$\log h(t \mid W) = g(t, W),$$

in which all covariates are modeled through the neural network and no separate linear coefficient is introduced. This extension was not part of the original FLEXI–Haz methodology and provides an additional option for settings in which flexible survival prediction is the primary objective rather than inference for specific covariate effects.

Beyond implementing the two model specifications, the project develops a complete estimation pipeline intended for practical use. The package provides input validation, administrative truncation, covariate standardization, time-grid and expanded-data construction, mini-batch neural-network training, survival prediction, post-training refinement of the linear coefficients, uncertainty estimation, and Optuna-based hyperparameter tuning through a unified and modular interface. Several computational and optimization improvements were also introduced. In particular, the partially linear implementation is approximately 20 times faster than the original research implementation, and a three-phase learning-rate schedule was incorporated to improve and accelerate convergence by progressively reducing the learning rate during training.

The contributions of this project can therefore be summarized as follows:

- A unified Python implementation supporting both the partially linear FLEXI–Haz model and a new fully nonparametric neural hazard formulation.
- A modular and user-friendly package that integrates the complete workflow from preprocessing and expanded-data construction through training, prediction, hyperparameter tuning, and statistical inference.
- A substantially more efficient implementation of the partially linear model, achieving an approximately 20× reduction in runtime relative to the original research code.
- Practical optimization improvements, including a three-phase learning-rate schedule, validation-based early stopping, coefficient refinement, and numerical stabilization mechanisms designed to improve convergence and robustness.
- An expanded empirical evaluation of FLEXI–Haz that considers both estimation and prediction. The partially linear and fully nonparametric implementations are compared with the neural partially linear proportional-hazards model of Zhong et al. (2022), the Cox proportional hazards model of Cox (1972), and Random Survival Forest of Ishwaran et al. (2008) in a controlled simulation study and in the Rotterdam breast cancer application.

The complete FLEXI–Haz Python implementation, including the partially linear and fully nonparametric models, reproducible simulation and real-data experiments, and package tests, is publicly available at GitHub.

The remainder of this report is organized as follows. Section 1 presents the statistical formulation of the partially linear and fully nonparametric FLEXI–Haz models and the likelihood used for estimation. Section 2 describes the Python implementation, including data expansion, model architecture, optimization, inference, prediction, and hyperparameter tuning. Section 3 presents the simulation and Rotterdam breast cancer experimental designs, and Section 4 reports the corresponding empirical results.

2 Methodological Formulation

Let U_i and C_i denote, respectively, the event time and censoring time for subject i, $i = 1, \ldots, n$. The observed data consist of n observations

$$O_i = (T_i, \Delta_i, X_i, Z_i), \qquad T_i = \min(U_i, C_i), \qquad \Delta_i = \mathbb{1}(U_i \leq C_i),$$

where T_i is the observed follow-up time and Δ_i indicates whether the event is observed. The vector $Z_i \in \mathbb{R}^p$ contains time-independent covariates whose effects are modeled through a finite-dimensional linear component, while $X_i \in \mathbb{R}^d$ contains time-independent covariates whose effects

are modeled flexibly through a neural-network component. We assume throughout that the observations are independent and identically distributed, that follow-up is restricted to a finite time horizon $\tau > 0$, and that censoring is conditionally independent of the event time given the covariates. That is,

$$U_i \perp C_i \mid (X_i, Z_i).$$

The at-risk process is denoted by

$$Y_i(t) = \mathbb{1}(T_i \geq t), \qquad 0 \leq t \leq \tau.$$

2.1 Partially Linear and Fully Nonparametric FLEXI–Haz Models

FLEXI–Haz of Ben Arie and Gorfine (2025) models the conditional hazard directly through a log-hazard function that combines time and covariate information. The partially linear version separates the covariates into two groups: $Z \in \mathbb{R}^p$, whose effects are modeled through a finite-dimensional linear parameter, and $X \in \mathbb{R}^d$, whose effects are modeled flexibly through a neural network. The conditional hazard is specified as

$$h_0(t \mid X, Z) = \exp\{\theta_0^\top Z + g_0(t, X)\}, \qquad 0 \leq t \leq \tau.$$

Equivalently,

$$\log h_0(t \mid X, Z) = \theta_0^\top Z + g_0(t, X).$$

Here, $\theta_0 \in \mathbb{R}^p$ is an interpretable vector of coefficients associated with the linear component, while $g_0(t, X)$ is an unknown flexible function that captures time dependence, nonlinear effects of X, and interactions between time and X.

Unlike the Cox-type proportional hazards model, FLEXI–Haz does not include a separate baseline hazard term. Instead, the general time pattern of the hazard is represented directly through the flexible component g_0. In the partially linear model, the effect of Z is linear and time-constant conditional on X, whereas the flexible component may vary with time. Therefore, the model retains an interpretable linear component while avoiding a fully proportional hazards structure.

The corresponding cumulative hazard and survival functions are

$$H_0(t \mid X, Z) = \int_0^t \exp\{\theta_0^\top Z + g_0(u, X)\}\, du, \qquad S_0(t \mid X, Z) = \exp\{-H_0(t \mid X, Z)\}.$$

Thus, once θ_0 and g_0 are estimated, the hazard, cumulative hazard, and survival function can be estimated by direct plug-in.

The fully nonparametric version removes the finite-dimensional linear component. In this case, all covariates are modeled through the flexible neural-network component. Let

$$W = (X^\top, Z^\top)^\top$$

denote the combined covariate vector. The fully nonparametric FLEXI–Haz model is

$$h_0(t \mid W) = \exp\{g_0(t, W)\}, \qquad 0 \leq t \leq \tau,$$

or equivalently,

$$\log h_0(t \mid W) = g_0(t, W).$$

Thus, the fully nonparametric model replaces the partially linear predictor $\theta_0^\top Z + g_0(t, X)$ by a single flexible component $g_0(t, W)$. This increases modeling flexibility, since all covariate effects

and time-covariate interactions may be learned by the neural network, but it removes the separate interpretable coefficient θ_0.

The cumulative hazard and survival functions in the fully nonparametric version are

$$H_0(t \mid W) = \int_0^t \exp\{g_0(u, W)\}\, du, \qquad S_0(t \mid W) = \exp\{-H_0(t \mid W)\}.$$

Thus, once g_0 is estimated, estimation of the hazard, cumulative hazard, and survival function is obtained by direct plug-in.

The two versions therefore share the same survival-analysis setting, censoring assumptions, likelihood principle, and numerical approximation strategy. Their main difference is whether part of the log-hazard is reserved for an interpretable finite-dimensional linear effect. The partially linear model is useful when inference on selected covariates is of primary interest, whereas the fully nonparametric model is mainly useful for flexible estimation of subject-specific hazard and survival functions.

2.2 Full-Likelihood Estimation

Estimation in both versions is based on the full likelihood for right-censored survival data. For a general conditional hazard function $h(t \mid \cdot)$, the individual likelihood contribution, up to constants that do not depend on the model parameters, is

$$\Delta_i \log h(T_i \mid \cdot) - \int_0^\tau Y_i(t) h(t \mid \cdot)\, dt.$$

For the partially linear FLEXI–Haz model, the empirical log-likelihood is

$$\ell_n(\theta, g) = \frac{1}{n} \sum_{i=1}^n \left[\Delta_i \{\theta^\top Z_i + g(T_i, X_i)\} - \int_0^\tau Y_i(t) \exp\{\theta^\top Z_i + g(t, X_i)\}\, dt \right].$$

For the fully nonparametric FLEXI–Haz model, the log-likelihood becomes

$$\ell_n(g) = \frac{1}{n} \sum_{i=1}^n \left[\Delta_i g(T_i, W_i) - \int_0^\tau Y_i(t) \exp\{g(t, W_i)\}\, dt \right].$$

Since $Y_i(t) = \mathbb{1}(T_i \geq t)$, the integrated hazard term can equivalently be written as

$$\int_0^\tau Y_i(t) h(t \mid \cdot)\, dt = \int_0^{\min(T_i, \tau)} h(t \mid \cdot)\, dt.$$

Thus, the likelihood only accumulates hazard over the time interval during which the subject is observed to be at risk.

In practice, the time integrals in the full likelihood generally do not have a closed-form solution, because the function g is unknown. Therefore, the integrals are approximated numerically. Let

$$0 = t_0 < t_1 < \cdots < t_m = \tau$$

be a finite partition of the follow-up interval $(0, \tau)$. In practice, the time grid is chosen to include observed event and censoring times. For each subject i, define the interval-specific event indicator

$$\Delta_{ij} = \mathbb{1}\{T_i \in (t_{j-1}, t_j],\ \Delta_i = 1\},$$

and the integration weight

$$w_{ij} = \min(T_i, t_j) - t_{j-1}, \qquad \text{for intervals satisfying } t_{j-1} < T_i.$$

The condition $t_{j-1} < T_i$ means that the subject is still at risk at the beginning of the interval $(t_{j-1}, t_j]$. The weight w_{ij} is the amount of time contributed by subject i inside that interval. For intervals fully before T_i, this weight equals $t_j - t_{j-1}$, while for the last observed interval it may be shorter.

For the partially linear FLEXI–Haz model, a Riemann-type approximation of the empirical log-likelihood is

$$\widetilde{\ell}_n(\theta, g) = \frac{1}{n} \sum_{i=1}^{n} \sum_{j: t_{j-1} < T_i} \left[\Delta_{ij} \{\theta^\top Z_i + g(t_j, X_i)\} - w_{ij} \exp\{\theta^\top Z_i + g(t_j, X_i)\} \right].$$

The FLEXI-Haz estimator is defined by

$$(\widetilde{\theta}, \widetilde{g}) \in \arg \max_{\theta \in \Theta,\ g \in \mathcal{G}} \widetilde{\ell}_n(\theta, g).$$

Equivalently, training can be performed by minimizing the negative discretized log-likelihood

$$\mathcal{L}_n(\theta, g) = -\widetilde{\ell}_n(\theta, g).$$

For the fully nonparametric FLEXI–Haz model, the corresponding approximated log-likelihood is

$$\widetilde{\ell}_n(g) = \frac{1}{n} \sum_{i=1}^{n} \sum_{j: t_{j-1} < T_i} \left[\Delta_{ij} g(t_j, W_i) - w_{ij} \exp\{g(t_j, W_i)\} \right],$$

and the FLEXI-Haz nonparametric estimator is

$$\widetilde{g} \in \arg \max_{g \in \mathcal{G}} \widetilde{\ell}_n(g),$$

or equivalently, the neural network is trained by minimizing

$$\mathcal{L}_n(g) = -\widetilde{\ell}_n(g) = \frac{1}{n} \sum_{i=1}^{n} \sum_{j: t_{j-1} < T_i} \left[w_{ij} \exp\{g(t_j, W_i)\} - \Delta_{ij} g(t_j, W_i) \right].$$

2.3 Interpretation and Inference

The two versions of FLEXI–Haz differ not only in their model structure, but also in the type of interpretation they provide. In the partially linear version, the parameter θ_0 is a finite-dimensional target that summarizes the effect of the covariates in Z, after adjusting for the flexible time-dependent and nonlinear effects represented by $g_0(t, X)$. Therefore, θ_0 can be interpreted as the log-hazard ratio associated with the covariates in Z, conditional on the flexible component.

More explicitly, for two subjects with the same X-covariates but different Z-covariates, Z_a and Z_b, the partially linear model implies

$$\frac{h_0(t \mid X, Z_a)}{h_0(t \mid X, Z_b)} = \exp\{\theta_0^\top (Z_a - Z_b)\}.$$

Thus, when X is held fixed, the effect of Z has a proportional-hazards interpretation. However, when subjects differ in X, the hazard ratio may also depend on time through the difference between their flexible components.

The partially linear formulation therefore supports inference for the finite-dimensional parameter θ_0. In particular, after estimating both θ_0 and g_0, one may construct confidence intervals and hypothesis tests for the components of θ_0. The theoretical results for FLEXI–Haz establish that the estimator of θ_0 is $\sqrt{n}$-consistent, asymptotically normal, and semiparametrically efficient under the model assumptions. The corresponding variance estimation is based on the efficient information for the linear component. In practice, this requires estimating an event-weighted projection of Z onto the nuisance space, which is implemented using additional neural networks. The resulting covariance estimate is then used to compute standard errors and confidence intervals for the components of θ_0. This is useful when the main scientific question concerns the effect of specific covariates while still requiring flexible adjustment for other covariates.

In contrast, the fully nonparametric version does not contain a separate finite-dimensional coefficient. Since all covariate effects are absorbed into $g_0(t, W)$, there is no parameter θ_0 to interpret or test. The main outputs of this model are estimates of the conditional hazard, cumulative hazard, and survival function. Therefore, the fully nonparametric version is mainly useful when the goal is flexible prediction or estimation of subject-specific survival patterns, rather than inference for a specific linear covariate effect. In this setting, a particularly important output is the estimated survival curve $\widehat{S}(t \mid W)$, for a subject with given covariates $W = (X^\top, Z^\top)^\top$, which is also provided by the implemented package.

3 Python Implementation

3.1 Overall Package Design

The package is organized into separate modules, each corresponding to a specific component of the estimation pipeline. This modular structure separates preprocessing, data expansion, model definition, optimization, prediction, inference, and tuning. As a result, the same implementation can support both the partially linear and fully nonparametric versions of FLEXI–Haz without duplicating the main training and prediction logic. Table 1 summarizes the main files in the package and their roles.

Module	Main role
`estimator.py`	Main estimator class, fitting procedure, prediction methods, and optional procedures for the linear parameter θ.
`networks.py`	Neural-network architecture for $g(t, X)$ and optional linear branch for $\theta^\top Z$.
`data.py`	Time-grid construction and expanded interval-level dataset construction.
`preprocessing.py`	Input validation, administrative truncation at τ, and covariate standardization.
`loss.py`	Negative discretized full log-likelihood used for model training.
`inference.py`	Variance estimation and confidence intervals for the linear parameter θ.
`tuning.py`	Optuna-based hyperparameter tuning for both model specifications.
`utils.py`	Device selection and random seed utilities.
`exceptions.py`	Custom exception classes for input validation and fitted-model checks.

Table 1: *Main modules in the implemented FLEXI–Haz package.*

The central module is `estimator.py`, which defines the main estimator class, `FlexiHazardModel`. This class coordinates the full fitting procedure: validation of the input data, truncation at the

administrative time horizon, standardization of covariates, construction of the time grid, expansion of the survival data into interval-level rows, neural-network training, optional post-training refinement of θ, optional inference for θ, and prediction of hazard, cumulative hazard, and survival functions.

The model architecture is implemented separately in `networks.py`. This module defines the flexible neural-network component for $g(t, X)$, and when the partially linear mode is used, an additional linear branch representing $\theta^\top Z$. In the fully nonparametric mode, the linear branch is omitted, and all covariates to be modeled are passed through the flexible input X. In terms of the notation from Section 2.1, this means that the combined covariate vector W is represented in the implementation by the input matrix X when Z is set to `None`.

The loss function is implemented in `loss.py`, while the construction of the discrete time grid and expanded-data representation is handled in `data.py`. This separation is important because both model specifications use the same expanded-data likelihood structure. The only difference is whether the log-hazard contains the additional linear term $\theta^\top Z$.

The package also includes modules for practical estimation utilities. The `preprocessing.py` module checks the validity of the survival data and standardizes covariates before training. The `inference.py` module implements procedures that are relevant only in the partially linear setting, namely variance estimation and confidence intervals for θ. The `tuning.py` module provides an Optuna-based interface for selecting architecture and optimization hyperparameters. Finally, `utils.py` and `exceptions.py` contain supporting utilities for reproducibility, device handling, and informative error reporting.

3.2 Input Validation, Truncation, and Standardization

Before constructing the expanded survival dataset, the package applies several preprocessing steps to ensure that the input data are valid and numerically suitable for neural-network training. These steps are implemented mainly in `preprocessing.py` and are called internally by `FlexiHazardModel` during fitting.

The first step is input validation. The observed time vector T is converted to a one-dimensional numeric array, and the event indicator Δ is checked to ensure that it contains only binary values. The covariate matrix X is converted to a two-dimensional numeric array. If the partially linear version is used, the matrix Z is validated in the same way. The package also checks that all arrays have the same number of observations, that all values are finite, that the observed times are nonnegative, and that the data contain at least one observed event. These checks are important because the likelihood cannot be fitted meaningfully when all observations are censored.

The second preprocessing step is administrative truncation at the time horizon τ. If no value of τ is supplied, the package uses the largest observed time. Otherwise, the supplied value of τ defines the maximal follow-up time used in fitting. Observed times larger than τ are truncated to τ, and events that occur after τ are treated as censored at τ. This matches the theoretical formulation, where the hazard is estimated only on a finite interval $[0, \tau]$.

The final preprocessing step is covariate standardization. When standardization is enabled, each column of X is centered and scaled before training. In the partially linear version, Z is standardized as well. Columns with nearly zero empirical standard deviation are assigned scale one, which prevents numerical instability from division by very small values. Standardization is useful because the neural-network component and the linear component are optimized jointly, and poorly scaled covariates can make gradient-based optimization less stable.

3.3 Time Grid and Expanded Dataset Construction

The discretized likelihood described in Section 2.2 requires a finite time grid and an expanded interval-level representation of the survival data. These steps are implemented in `data.py`. The purpose of this module is to transform the original subject-level data into rows corresponding to time intervals in which each subject is at risk.

The time grid is constructed on the interval $[0, \tau]$. In the default configuration, `grid_size="n"` and `include_observed_times=True`. This means that the package first creates a uniform grid with n points, where n is the number of observed subjects, and then augments this grid with all observed follow-up times. After construction, the grid is sorted and duplicate time points are removed. Including observed times in the grid helps ensure that event and censoring times are represented accurately in the discretized likelihood, while the additional uniform grid provides coverage of the full follow-up interval. The choice of grid size creates a tradeoff between numerical accuracy and computational complexity. A finer grid gives a more accurate approximation of the continuous-time likelihood, but it also increases the number of expanded rows, which leads to higher memory usage, longer training time, and larger storage requirements for the expanded dataset.

Given the time grid

$$0 = t_0 < t_1 < \cdots < t_m = \tau,$$

each subject is expanded into one row for every interval $(t_{j-1}, t_j]$ satisfying $t_{j-1} < T_i$. Each expanded row contains the right endpoint of the interval t_j, the covariates of subject i, the interval-level event indicator Δ_{ij}, the integration weight w_{ij}, and the original subject index. In the partially linear version, both X_i and Z_i are stored, whereas in the fully nonparametric version all covariates are supplied through X_i.

Equivalently, the original subject-level observation

$$O_i = (T_i, \Delta_i, X_i, Z_i)$$

is transformed into multiple interval-level rows. In the partially linear version, the expanded rows have the form

$$R_{ij}^{\mathrm{PL}} = (t_{j-1}, t_j, X_i, Z_i, \Delta_{ij}, w_{ij}, i), \qquad j : t_{j-1} < T_i,$$

whereas in the fully nonparametric version they have the form

$$R_{ij}^{\mathrm{NP}} = (t_{j-1}, t_j, X_i, \Delta_{ij}, w_{ij}, i), \qquad j : t_{j-1} < T_i.$$

The event indicator is redefined as

$$\Delta_{ij} = \mathbb{1}\{T_i \in (t_{j-1}, t_j],\ \Delta_i = 1\}.$$

Thus, $\Delta_{ij} = 1$ only for the interval containing an observed event time, and it is zero for all other intervals. The integration weight is defined as

$$w_{ij} = \min(T_i, t_j) - t_{j-1}, \qquad \text{for intervals satisfying } t_{j-1} < T_i.$$

This weight represents the amount of time contributed by subject i inside the interval $(t_{j-1}, t_j]$. For intervals fully before T_i, the weight equals $t_j - t_{j-1}$, while for the last observed interval it may be shorter.

Using the expanded rows, the discretized likelihood can be written as a sum of row-level contributions. For the partially linear model, the negative row-level contribution is

$$w_{ij} \exp\{\theta^\top Z_i + g(t_j, X_i)\} - \Delta_{ij}\{\theta^\top Z_i + g(t_j, X_i)\}.$$

For the fully nonparametric model, where all covariates are represented by W_i, the corresponding row-level contribution is

$$w_{ij} \exp\{g(t_j, W_i)\} - \Delta_{ij} g(t_j, W_i).$$

Summing these row-level terms over the expanded dataset gives the negative discretized log-likelihood from Section 2.2. This representation is convenient computationally because it converts the survival likelihood into a standard mini-batch optimization problem, where each mini-batch contains expanded interval-level rows rather than complete subject-level trajectories.

The expanded dataset also stores the original subject index for each row. This is important because the expanded rows are not statistically independent at the row level: several rows may come from the same subject. Therefore, train-validation splitting is performed at the subject level and then transferred to the corresponding expanded rows. This avoids data leakage between training and validation, prevents the model from being evaluated on interval rows belonging to subjects it has already seen during training, and reduces the risk of overfitting being hidden by an artificially low validation loss.

The expanded dataset stores the interval endpoints t_j and the interval lengths w_{ij} on the original time scale. However, before an interval endpoint is passed to the neural-network component, the estimator optionally rescales it by the administrative time horizon τ. In the default setting, `scale_time=True`. This means that the network receives $t_j/\tau \in [0, 1]$ rather than the raw value t_j. This scaled time input is used to improve numerical conditioning and make gradient-based optimization more stable, which can help training converge more reliably. However, the interval weights w_{ij} remain on the original time scale, so the discretized likelihood still approximates the continuous-time likelihood over $[0, \tau]$.

3.4 Model Architecture

The neural-network architecture is implemented in `networks.py`. The main class is `FlexiHazardNet`, which represents the log-hazard predictor used by the estimator. The model contains a feedforward neural network for the flexible component and, in the partially linear version, an additional linear branch for the covariates whose effects are modeled parametrically.

In both versions of FLEXI–Haz, the flexible component of the log-hazard is approximated by a neural network. Let g_ω denote this neural-network approximation, where ω is the collection of network weights and biases. The network is implemented as a multilayer perceptron. For an input vector u, a typical fully connected architecture can be written as

$$g_\omega(u) = W_K \sigma\left(W_{K-1} \sigma\left(\cdots \sigma(W_0 u + b_0)\right) + b_{K-1}\right) + b_K,$$

where σ is an activation function applied componentwise. The package supports several activation functions, including ReLU, ELU, tanh, and GELU. The depth and width of the network are controlled by the list of hidden-layer sizes supplied by the user. By default, the package uses `hidden_layers=(64,64)`, and `activation="relu"`. Thus, unless changed by the user, the flexible component is represented by a two-hidden-layer ReLU network with 64 hidden units in each layer.

In the partially linear version, the neural network approximates the flexible component $g_0(t, X)$. The input to the network is the concatenation of the scaled time coordinate and the covariate vector X:

$$u = (t/\tau, X^\top)^\top.$$

Thus, if $X \in \mathbb{R}^d$, the input dimension of the neural network is $d + 1$. The covariates Z are not passed through this flexible network. Instead, they enter the model through a separate linear branch

without an intercept, which represents the scalar linear predictor $\theta^\top Z$. The full implemented log-hazard predictor is therefore

$$\eta_\omega(t, X, Z) = \theta^\top Z + g_\omega(t/\tau, X).$$

The absence of an intercept in the linear branch is intentional, because the baseline time pattern and general log-hazard level are already represented by the flexible component $g_\omega(t/\tau, X)$.

In the fully nonparametric version, no linear branch is created. All covariates are supplied through the flexible input matrix. In terms of the notation from Section 2.1, if $W = (X^\top, Z^\top)^\top$ denotes the combined covariate vector, then W is passed to the implementation as the covariate matrix X, with $Z = \texttt{None}$. The network input is then

$$u = (t/\tau, W^\top)^\top,$$

and the full log-hazard predictor is

$$\eta_\omega(t, W) = g_\omega(t/\tau, W).$$

Thus, the same network class supports both model specifications, with the presence or absence of the linear branch determining whether the fitted model is partially linear or fully nonparametric.

In theoretical analyses, the neural-network function class is usually controlled by restrictions such as boundedness, sparsity, or architectural constraints. These restrictions control the statistical complexity of the estimator. In the practical implementation, exact sparsity constraints are not imposed explicitly. Instead, the effective complexity of the fitted network is controlled through hyperparameter choices, including depth, width, learning rate, batch size, dropout rate, and early-stopping patience.

The network returns both the full log-hazard predictor and the flexible component. The full predictor is needed for likelihood optimization and hazard prediction. The flexible component alone is useful in partially linear procedures that operate on $g_\omega(t/\tau, X)$ separately, such as post-training refinement and inference for the linear coefficient θ.

3.5 Training and Optimization

Model training is implemented in `estimator.py` and is performed after preprocessing, time-grid construction, expanded-data construction, and network initialization. The estimator trains the neural hazard model on the expanded interval-level dataset using mini-batch gradient-based optimization. In the partially linear version, the parameters of the neural-network component g_ω and the linear coefficient θ are optimized jointly during the main training stage. In the fully nonparametric version, only the parameters of the neural network are optimized.

The optimization procedure uses the Adam optimizer. At each epoch, the expanded training rows are divided into mini-batches, the model evaluates the log-hazard predictor for the rows in the batch, and the negative discretized log-likelihood is minimized by backpropagation. The default optimization settings are

$$\texttt{learning_rate} = 10^{-3}, \qquad \texttt{batch_size} = 32^3 = 32768, \qquad \texttt{max_epochs} = 500.$$

It is important to note that `batch_size` refers to the number of rows in the expanded interval-level dataset, rather than to the number of subjects in the original dataset. Under the default expansion, where the time grid contains n grid points, the number of expanded rows grows on the order of n^2. Consequently, the expanded dataset can be substantially larger than the original subject-level dataset, and relatively large mini-batches are typically appropriate. The default value

was chosen with an original sample size of approximately $n = 8000$ subjects in mind. Users should therefore treat this value as a starting point and adjust it according to the size of their dataset and the available computational resources.

For numerical stability, the log-hazard values are clipped before the exponential term is computed. This is important because very large positive log-hazard values can cause numerical overflow in the exponential function, whereas very large negative values can lead to extremely small gradients. This clipping step therefore stabilizes training without changing the structure of the optimization objective.

Validation is used to monitor training and select the final fitted model. The train-validation split is performed at the subject level and then mapped to the corresponding expanded rows. This prevents rows from the same subject from appearing in both the training and validation sets. The default setting is `validation_fraction`=0.33, which means that one third of the subjects are held out for validation. During each epoch, the training loss is computed from mini-batches, while the validation loss is computed on the held-out expanded rows. The estimator stores the network state that achieves the lowest validation loss and restores this state at the end of training. Therefore, the final fitted model is the model with the best validation performance, not necessarily the model from the last epoch. Regularization during training is mainly controlled through validation-based early stopping and through the optimization hyperparameters chosen by the user. The package also supports explicit neural-network regularization through dropout, although it is disabled in the default configuration. Specifically, the default values are

$$\texttt{dropout} = 0.0, \qquad \texttt{patience} = 30.$$

Therefore, by default, training stops if the validation loss does not improve for 30 consecutive epochs. This provides a form of regularization through model selection along the optimization path, even when no explicit dropout is applied.

The default training procedure uses a three-phase learning-rate schedule, controlled by `three_phase_lr=True`. If the base learning rate is denoted by α, the model is trained sequentially using the three learning rates

$$\alpha, \qquad 0.1\alpha, \qquad 0.01\alpha.$$

With the default value $\alpha = 10^{-3}$, the three phases therefore use learning rates

$$10^{-3}, \qquad 10^{-4}, \qquad 10^{-5}.$$

Each phase runs the same mini-batch training procedure with Adam, early stopping, and validation-loss monitoring. This schedule allows the model to first make relatively large optimization steps and then continue training with smaller learning rates, which can improve convergence and stabilize the final parameter estimates.

The package records the training and validation losses from all learning-rate phases, together with the best validation epoch. These diagnostics are stored in the fitted estimator and can be used to inspect the optimization trajectory, compare training behavior across model configurations, and identify whether training stopped because of early stopping or because the maximum number of epochs was reached.

3.6 Linear Coefficient Initialization, Refinement, and Inference

Some procedures in the package are relevant only for the partially linear version of FLEXI–Haz, where the log-hazard includes the finite-dimensional linear term $\theta^\top Z$. These procedures are implemented mainly in `estimator.py` and `inference.py`. They are not used in the fully nonparametric version, since in that case the model does not contain a separate linear parameter θ.

When the partially linear model is fitted, the package initializes the linear coefficient according to the argument `theta_init`. The default setting is `theta_init="cox"`, meaning that the linear coefficient is initialized using a Cox proportional hazards model fitted on the covariates Z. This provides a reasonable starting point for the linear branch before joint neural-network optimization begins. If standardization is applied to Z, the Cox coefficient is transformed to the standardized scale so that it matches the covariates used by the implemented network. If Cox initialization is disabled or cannot be computed successfully, the package falls back to an alternative initializations – random initialization (`theta_init="random"` or `theta_init="none"`), or zero initialization (`theta_init="zero"`)

After the main neural-network training stage, the package can optionally refine the estimate of θ. This option is enabled by default through `refine_theta=True`. In this step, the fitted flexible component $\widehat{g}(t, X)$ is held fixed, and the linear coefficient is updated by optimizing the likelihood with respect to θ alone. This post-training refinement is intended to improve the finite-dimensional estimate after the nuisance function has already been learned. Since this step targets only the linear component, it is relevant only when Z is supplied.

When `refine_theta=True`, the timing of the refinement step depends on the learning-rate schedule used during training. Under the default setting `three_phase_lr=True`, the model is trained sequentially with three learning rates, and the refinement of θ is applied after each learning-rate phase. In each refinement step, the fitted neural-network component is held fixed, and the likelihood is optimized only with respect to the linear coefficient θ. If `three_phase_lr=False`, the model is trained using only the base learning rate, and the optional refinement of θ is applied once after the main training stage.

The package also contains an inference module for the linear coefficient. Variance estimation is enabled by default through `compute_theta_variance=True`. The purpose of this module is to estimate uncertainty for $\widehat{\theta}$ after fitting the partially linear model. The inference procedure uses the fitted model and the training data to estimate a covariance matrix for $\widehat{\theta}$. The resulting standard errors can then be used to construct confidence intervals and summary tables for the components of θ.

The variance estimation procedure relies on auxiliary projection networks. These networks are controlled by the default settings

$$\texttt{projection_max_epochs} = 300, \qquad \texttt{projection_patience} = 30, \qquad \texttt{projection_n_splits} = 5.$$

Thus, by default, the projection networks are trained for at most 300 epochs, use early stopping with patience 30, and are fitted using five data splits. The projection networks receive the same scaled time input as the main neural-network component, ensuring consistency between training and inference.

The user-facing method `theta_summary` reports the estimated coefficient, its standard error, and a confidence interval for each component of θ. This output is meaningful only in the partially linear setting, where Z represents covariates whose effects are modeled linearly. In the fully nonparametric setting, all covariate effects are absorbed into the neural network, so there is no finite-dimensional coefficient for which such a summary can be produced.

3.7 Prediction Functions

After fitting, the package provides several prediction methods through the main `FlexiHazardModel` class. These methods use the trained neural network, the fitted preprocessing transformations, and the selected time grid to estimate different survival quantities. The prediction interface is designed to support both the partially linear and fully nonparametric versions of the model.

The most basic prediction method is `predict_g`, which returns the estimated flexible component of the log-hazard. In the partially linear version, this corresponds to

$$\widetilde{g}(t/\tau, X),$$

while the full log-hazard also includes the additional linear term $\widetilde{\theta}^\top Z$. In the fully nonparametric version, the flexible component is the entire log-hazard, since no separate linear branch is included.

The method `predict_hazard` returns the estimated hazard function. For the partially linear version, the estimated hazard is

$$\widetilde{h}(t \mid X, Z) = \exp\{\widetilde{\theta}^\top Z + \widetilde{g}(t/\tau, X)\}.$$

For the fully nonparametric version, the estimated hazard is

$$\widetilde{h}(t \mid X) = \exp\{\widetilde{g}(t/\tau, X)\}.$$

As in the fitting stage, all covariates should be passed through X when the fully nonparametric version is used.

The package also provides methods for estimating the cumulative hazard and survival function. The method `predict_cumulative_hazard` computes the estimated cumulative hazard using a Riemann-type sum over the requested prediction times:

$$\widehat{H}(t \mid \cdot) = \sum_{j: t_j \le t} (t_j - t_{j-1})\, \widehat{h}(t_j \mid \cdot).$$

The method `predict_survival` then returns

$$\widehat{S}(t \mid \cdot) = \exp\{-\widehat{H}(t \mid \cdot)\}.$$

Thus, the survival prediction is obtained by first estimating the hazard, then numerically integrating it to obtain the cumulative hazard, and finally applying the standard survival transformation.

Prediction times are supplied by the user on the original time scale. Internally, these times are scaled by the fitted value of τ before being passed to the neural network, consistently with the training procedure. However, the cumulative hazard calculation uses the original prediction times and original time-grid differences when performing numerical integration. Thus, the returned hazard, cumulative hazard, and survival curves are reported as functions of the original time scale.

Before prediction, new covariate values are transformed using the same standardization parameters that were fitted on the training data. This ensures that the model receives covariates on the same scale during training and prediction. In the partially linear case, both X and Z are transformed. In the fully nonparametric case, only the covariate matrix passed through X is transformed.

3.8 Hyperparameter Tuning

The package includes an Optuna-based tuning interface implemented in `tuning.py`. The function `tune_flexihaz` searches over candidate architecture, optimization, and time-grid configurations for both the partially linear and fully nonparametric versions of FLEXI–Haz. The model specification is determined by whether Z is supplied: providing Z tunes the partially linear model, whereas setting $Z =$ `None` tunes the fully nonparametric model.

The default search space is

```
default_space = {
    "hidden_layers": [
        [32], [64], [128], [64, 64],
        [128, 64], [128, 128], [256, 128]
    ],
    "learning_rate": (1e-4, 5e-2),
    "dropout": (0.0, 0.0),
    "batch_size": [32 ** 3, 16 ** 4, 24 ** 4, 32 ** 4],
    "activation": ["relu"],
    "grid_size": ["n"],
}
```

Thus, the default procedure compares several one-layer and two-layer neural-network architectures and searches over the learning rate. Dropout, activation, and grid size are fixed by default because their corresponding search spaces contain only one value or a degenerate interval. The batch size is selected from the supplied list of candidate values.

The hyperparameters are sampled according to their statistical type. The neural-network architecture `hidden_layers` is treated as a categorical choice. Internally, Optuna samples the index of one candidate architecture and then converts this index to the corresponding tuple of hidden-layer widths. The parameters `batch_size`, `activation`, and `grid_size` are also sampled categorically from the lists supplied in the search space.

The learning rate is specified as a tuple

$$(\alpha_{\min}, \alpha_{\max}),$$

and is sampled as a continuous floating-point value on a logarithmic scale. Therefore, the default interval $(10^{-4}, 5\times10^{-2})$ searches across several orders of magnitude rather than sampling uniformly on the original numerical scale. If the lower and upper endpoints are equal, the learning rate is held fixed at that value.

The dropout search space is also specified as a tuple

$$(d_{\min}, d_{\max}),$$

but it is sampled uniformly on the ordinary linear scale. If the two endpoints are equal, dropout is fixed. Consequently, the default value `dropout=(0.0,0.0)` disables dropout throughout tuning.

A custom search space can be supplied through the `search_space` argument. The provided entries replace the corresponding defaults, while unspecified entries retain their default values. A custom space must therefore use the following forms:

- `hidden_layers`: a list of candidate architectures, where each architecture is a list of hidden-layer widths;
- `learning_rate`: a tuple `(low, high)`, sampled logarithmically;
- `dropout`: a tuple `(low, high)`, sampled uniformly on the linear scale;
- `batch_size`: a list of categorical candidate batch sizes;
- `activation`: a list of categorical activation-function names;
- `grid_size`: a list of categorical grid-size specifications;

For example, a user who wishes to compare two architectures, tune dropout, and consider two time-grid sizes could define

```
custom_space = {
    "hidden_layers": [[64, 64], [128, 128]],
    "learning_rate": (1e-4, 1e-2),
    "dropout": (0.0, 0.3),
    "batch_size": [8192, 32768],
    "activation": ["relu", "gelu"],
    "grid_size": ["n", 200],
}
```

For each Optuna trial, the sampled configuration is used to construct and fit a `FlexiHazardModel`. The trial objective is the validation negative log-likelihood, which Optuna minimizes. When `folds=1`, evaluation uses the internal subject-level train-validation split of the estimator, and the minimum validation loss achieved during training is returned. When `folds`=K and $K > 1$, the observations are divided into K subject-level folds, a separate model is fitted for each training fold, and the mean validation loss across the held-out folds is returned.

The administrative time horizon is kept fixed across all folds. If the user does not supply τ, the largest observed follow-up time in the full dataset is used. Keeping a common time horizon prevents different folds from being evaluated over different follow-up intervals.

The tuning procedure uses Optuna's tree-structured Parzen estimator sampler. By default, pruning is enabled using a median pruner with five startup trials and ten warm-up steps. During training, the current validation loss is reported to Optuna after each epoch. A poorly performing trial may therefore be stopped before reaching its maximum number of epochs. Since FLEXI–Haz training can be computationally expensive for large grids or wide neural networks, pruning is useful for concentrating computation on more promising configurations.

The tuning routine focuses only on selecting a predictive model configuration. To reduce computational cost, linear-coefficient refinement and variance estimation are disabled during the trials. In particular, the trial models use

`refine_theta=False`, `compute_theta_variance=False`.

The maximum number of epochs used in each trial is controlled separately by the `max_epochs` argument of `tune_flexihaz`, whose default value is 120, and trial models use an early-stopping patience of 15. After suitable hyperparameters are selected, the final model can be refitted under the chosen configuration, and optional procedures such as refinement and inference for θ can then be applied in the partially linear setting.

3.9 User Interface and Example Usage

Although the package contains several internal modules, the main user-facing interface is simple. Most users interact with the package through the `FlexiHazardModel` class and, optionally, through the `tune_flexihaz` function. This subsection gives representative examples of how the two model versions are fitted and used for prediction.

The partially linear version is fitted by supplying both X and Z. The covariates in X enter the flexible neural-network component, while the covariates in Z enter the linear component.

```
1 from flexihaz import FlexiHazardModel
2
```

```
model = FlexiHazardModel(
    tau=tau,
    hidden_layers=(64, 64),
    learning_rate=1e-3,
    three_phase_lr=True,
    batch_size=16 ** 4,
    max_epochs=500,
    patience=30,
    standardize=True,
    refine_theta=True,
    compute_theta_variance=True,
    scale_time=True,
    seed=1,
)

model.fit(T, delta, X, Z=Z)
```

Listing 1: Fitting the partially linear FLEXI–Haz model.

The fully nonparametric version is fitted by setting $Z =$ `None`. In this case, all covariates that should be modeled by the neural network are supplied through X. If the theoretical covariate vector is $W = (X^\top, Z^\top)^\top$, then in the implementation W is passed as the argument `X`.

```
import numpy as np
from flexihaz import FlexiHazardModel

W = np.column_stack([X, Z])

model_np = FlexiHazardModel(
    tau=tau,
    hidden_layers=(64, 64),
    learning_rate=1e-3,
    three_phase_lr=True,
    batch_size=16 ** 4,
    max_epochs=500,
    patience=30,
    standardize=True,
    scale_time=True,
    seed=1,
)

model_np.fit(T, delta, W, Z=None)
```

Listing 2: Fitting the fully nonparametric FLEXI–Haz model.

After fitting, the model can be used to estimate hazard, cumulative hazard, and survival curves on a chosen prediction grid. The prediction times are provided on the original time scale. Internally, the model applies the same time scaling used during training before evaluating the neural network.

```
time_grid = np.linspace(0.0, tau, 100)

survival = model.predict_survival(
    X_new=X_test,
    times=time_grid,
    Z_new=Z_test,
)
```

Listing 3: Predicting survival curves from the partially linear model.

For the fully nonparametric model, prediction uses the combined covariate matrix and does not supply a separate Z-matrix.

```
W_test = np.column_stack([X_test, Z_test])

survival_np = model_np.predict_survival(
    X_new=W_test,
    times=time_grid,
    Z_new=None,
)
```

Listing 4: Predicting survival curves from the fully nonparametric model.

When the partially linear model is fitted, the package can also return a summary for the linear coefficient. This includes the estimated coefficient and, when variance estimation was requested, standard errors and confidence intervals.

```
theta_table = model.theta_summary(alpha=0.05)
print(theta_table)
```

Listing 5: Summary of the linear coefficient in the partially linear model.

Finally, hyperparameters can be selected using the Optuna-based tuning function. The same interface can be used for both versions of the model: supplying Z tunes the partially linear version, while setting $Z = \texttt{None}$ tunes the fully nonparametric version.

```
from flexihaz import tune_flexihaz

tuning_result = tune_flexihaz(
    T=T,
    delta=delta,
    X=X,
    Z=Z,
    tau=tau,
    n_trials=30,
    folds=1,
    max_epochs=120,
    seed=1,
)

best_params = tuning_result["best_params"]
best_value = tuning_result["best_value"]
```

Listing 6: Optuna-based hyperparameter tuning.

4 Experimental Design

4.1 Simulation Study

We conducted a simulation study based on the continuous-covariate setting used in Ben Arie and Gorfine (2025), while extending the comparison to include additional survival-learning methods and predictive performance measures. We considered nuisance covariates $X = (X_1, X_2, X_3)^\top$ and primary covariates $Z = (Z_1, Z_2)^\top$, with all five covariates generated independently from Unif$[-1, 1]$. The true coefficient vector was

$$\theta_0 = (2, -1)^\top,$$

and the nonlinear nuisance function was

$$f(X) = 0.2(X_1 + X_2) + 0.5X_1X_2 + X_3^2.$$

Event times were generated from the hazard

$$h_0(t \mid X, Z) = 0.1 \exp\left\{\left(0.1 + f(X)^2\right) t + \theta_0^\top Z\right\}, \qquad t \in [0, \tau],$$

with administrative study horizon $\tau = 10$. Since the effect of X enters through the interaction between $f(X)^2$ and time, the full covariate effect is non-proportional over time. Event times were sampled by inverse-transform sampling from the corresponding survival function. Independent random censoring times were generated from an exponential distribution with mean 30, equivalently rate 1/30, and administrative censoring was imposed at $\tau = 10$. Thus, for each subject,

$$T_i = \min\{U_i, C_i, \tau\}, \qquad \Delta_i = \mathbf{1}\{U_i \le C_i,\ U_i \le \tau\}.$$

The simulation consisted of 200 independently generated training datasets, each containing $n = 8000$ subjects. In addition, a single independent test sample of size 10,000 was generated and kept fixed throughout all replications and model comparisons. This common test set ensured that differences in predictive performance were not driven by variation in the evaluation sample. Because the data-generating mechanism is known, the true cumulative hazard and survival probability of every test subject could be evaluated analytically. Survival probabilities were evaluated at

$$0.25,\ 0.50,\ \ldots,\ 10.00,$$

while censoring-adjusted prediction metrics were evaluated at the corresponding grid points strictly below τ.

We compared five methods. The first was the partially linear FLEXI–Haz model,

$$\log h(t \mid X, Z) = \theta^\top Z + g(t, X),$$

which retains Z as the interpretable linear component and models the joint effect of time and X through the neural network. The second was a fully nonparametric FLEXI–Haz model, in which the five covariates $(X^\top, Z^\top)^\top$ were all supplied to the flexible neural-network component and no separate linear term was used. The third method was the partially linear neural proportional-hazards model of Zhong et al. (2022), which preserves a separate linear component for Z but restricts the nuisance effect to a proportional-hazards structure. We additionally fitted a standard Cox proportional hazards model of Cox (1972) using all five covariates linearly, and a Random Survival Forest of Ishwaran et al. (2008) using all five covariates as flexible predictors.

Hyperparameters for the three neural-network models, namely partially linear FLEXI–Haz, fully nonparametric FLEXI–Haz, and the model of Zhong et al. (2022), were selected once using

the first simulated training dataset and were then held fixed across all 200 replications. To ensure a fair comparison, the same hyperparameter search space and tuning protocol were used for all three models. Tuning was performed with Optuna using 100 trials and five-fold cross-validation. The candidate architectures included one to five hidden layers with several combinations of widths between 16 and 256 units, while the learning rate was searched over the interval

$$[10^{-3}, 5 \times 10^{-2}].$$

ReLU activation and a batch size of 100,000 were used throughout. Training was performed for at most 2000 epochs, with 33% of the training data reserved for validation and early stopping applied with patience 30. For the two FLEXI–Haz models, the time-grid specification was fixed at `grid_size="n"`, and no dropout was applied.

The partially linear FLEXI–Haz model selected three hidden layers with widths

$$(32, 32, 64)$$

and learning rate

$$1.187 \times 10^{-3}.$$

The fully nonparametric FLEXI–Haz model selected four hidden layers with 64 units each and learning rate

$$1.387 \times 10^{-3}.$$

The Zhong et al. (2022) model selected four hidden layers with 64 units each and learning rate

$$2.463 \times 10^{-2}.$$

For Random Survival Forest, structural hyperparameters were tuned once on the first training replication using Optuna with three-fold cross-validation and mean integrated Brier score as the optimization criterion. Tuning was performed using 200 trees, after which the final models used 500 trees. The selected configuration had

$$\begin{aligned} \texttt{min_samples_leaf} &= 10, \\ \texttt{min_samples_split} &= 20, \\ \texttt{max_features} &= 0.5, \\ \texttt{max_depth} &= 20, \\ \texttt{max_samples} &= 0.9. \end{aligned}$$

The standard Cox model was fitted without penalization and did not require hyperparameter tuning.

Predictive performance was assessed using the integrated Brier score, the mean Brier score across evaluation times, time-dependent AUC and its mean over time, the concordance index, and direct comparison between the estimated and true survival functions. Since the true survival probability is known in the simulation, we additionally calculated the mean survival bias, mean absolute bias, and root mean squared error across evaluation times. The implementation uses inverse-probability-of-censoring weighted survival metrics and retains only evaluation times that lie within the observed follow-up range and contain both subjects experiencing an event by that time and subjects remaining at risk afterward.

For methods that estimate a separate linear coefficient, namely partially linear FLEXI–Haz and Zhong et al. (2022), we additionally evaluated estimation and inference for θ_0. Across the 200 replications, we recorded the mean coefficient estimate, its bias, the empirical standard deviation of the estimates, the mean estimated standard error, and empirical coverage of the nominal 90% and 95% confidence intervals. The resulting simulation outcomes are presented separately in Section 5.1.

4.2 Real-data Application: Rotterdam Breast Cancer Study

To evaluate FLEXI–Haz in a real-data setting, we analyzed the Rotterdam Breast Cancer dataset, available in the `survival` package in R and also considered in Ben Arie and Gorfine (2025). The outcome was defined as the time from primary surgery to the earlier of breast cancer recurrence or death. Thus, for each patient,

$$T_i = \min\{\texttt{rtime}_i, \texttt{dtime}_i\},$$

with the event indicator equal to one if either recurrence or death was observed. The dataset contains $n = 2982$ patients and nine baseline covariates: age, progesterone receptor level, estrogen receptor level, number of positive lymph nodes, menopausal status, tumor size, tumor grade, chemotherapy, and hormonal therapy.

The primary objective was to investigate whether the effects of chemotherapy and hormonal therapy differ according to estrogen receptor (ER) level. We therefore used an ER-interaction design, dividing patients into low- and high-ER groups using the threshold $ER = 61$. The linear component was defined as

$$Z = \begin{pmatrix} \texttt{hormon}\,\mathbf{1}\{\mathrm{ER} < 61\} \\ \texttt{hormon}\,\mathbf{1}\{\mathrm{ER} \geq 61\} \\ \texttt{chemo}\,\mathbf{1}\{\mathrm{ER} < 61\} \\ \texttt{chemo}\,\mathbf{1}\{\mathrm{ER} \geq 61\} \end{pmatrix}.$$

This formulation yields separate treatment-effect estimates for hormonal therapy and chemotherapy in the low-ER and high-ER groups. The flexible covariate component contained age, number of positive lymph nodes, progesterone receptor level, estrogen receptor level, menopausal status, tumor grade, and tumor size. Tumor size was represented using one-hot encoding, whereas the remaining variables retained their numerical representations.

We compared the same five survival models considered in Section 4.1: partially linear FLEXI–Haz, fully nonparametric FLEXI–Haz, the partially linear neural proportional-hazards model of Zhong et al. (2022), a standard Cox proportional hazards model of Cox (1972), and a Random Survival Forest of Ishwaran et al. (2008). For partially linear FLEXI–Haz and the Zhong et al. (2022) model, the four ER-specific treatment terms were included in the linear component, while the remaining clinical variables were modeled through the flexible component. For fully nonparametric FLEXI–Haz, all treatment and clinical covariates were combined and modeled jointly through the neural network. Random Survival Forest similarly used the complete covariate vector without imposing a separate linear structure.

For the two FLEXI–Haz models, the model of Zhong et al. (2022), and Random Survival Forest, predictive performance was evaluated using five outer folds generated once and shared across methods. Within each outer training fold, hyperparameters were tuned using an internal 80/20 training-validation split, and the selected configuration was then used to fit the model for evaluation on the held-out outer fold.

The same neural-network hyperparameter search space was used for partially linear FLEXI–Haz, fully nonparametric FLEXI–Haz, and the Zhong et al. (2022) model. The candidate architectures consisted of one to five hidden layers with several combinations of widths between 16 and 256 units, and the learning rate was searched over

$$[10^{-3}, 5 \times 10^{-2}].$$

ReLU activation and a batch size of 100,000 were used throughout. Hyperparameter selection was performed with Optuna using 100 trials within each outer fold, with a maximum of 2000 epochs for each tuning fit. The FLEXI–Haz models additionally used `grid_size="n"`, with dropout set

to zero. Final neural-model training was performed for at most 2000 epochs with validation-based early stopping. For partially linear FLEXI–Haz, coefficient refinement and covariance estimation were enabled, and the projection networks used for inference were also trained for at most 2000 epochs with five-fold cross-fitting.

Random Survival Forest was tuned separately within each outer training fold using Optuna with an internal 80/20 training-validation split. The tuning objective was the validation integrated Brier score. The search covered the minimum numbers of samples required for a split and a terminal leaf, the number or proportion of candidate features considered at each split, maximum tree depth, and the fraction of observations sampled for each tree. The tuning models used 100 trees, and each final forest was fitted with 500 trees using the selected structural hyperparameters.

The standard Cox proportional hazards model was fitted once to all 2982 patients using the same ER-interaction design, without applying the five-fold splitting procedure.

Predictive performance was assessed on a time grid with 50-day spacing over the observed follow-up period. The evaluation included the integrated Brier score, mean Brier score, time-dependent AUC, mean AUC across evaluation times, and the concordance index. In addition to prediction, treatment-effect estimation was examined for partially linear FLEXI–Haz, Zhong et al. (2022) model, and Cox PH by reporting the estimated ER-specific log-hazard coefficients, standard errors, 95% confidence intervals, and corresponding hazard ratios. The resulting estimates and predictive performance are presented in Section 5.1.

5 Results

5.1 Simulation Study Results

Table 2 summarizes estimation and inference for the two linear coefficients under the partially linear FLEXI–Haz model and the model of Zhong et al. (2022). The partially linear FLEXI–Haz estimates were close to the true values for both coefficients. For $\theta_1 = 2$, the mean estimate was 1.993, with bias -0.007, while for $\theta_2 = -1$, the mean estimate was -0.993, with bias 0.007. The empirical standard deviations were also close to the corresponding mean estimated standard errors. The empirical coverage probabilities of the 95% confidence intervals were 0.945 for both coefficients, close to the nominal level.

In contrast, the Zhong et al. (2022) model exhibited substantial bias in both coefficients. The mean estimates were 1.790 and -0.900, corresponding to biases of -0.210 and 0.100, respectively. Although its estimated standard errors were similar in magnitude to the empirical standard deviations, the bias was large relative to these uncertainties. Consequently, the 95% coverage probabilities were 0.000 for θ_1 and 0.055 for θ_2. These results indicate that, in the non-proportional hazards setting considered here, the proportional-hazards restriction of the Zhong et al. (2022) model leads to biased estimation of the linear effects, whereas FLEXI–Haz retains accurate estimation and approximately nominal inference.

Prediction performance across the 200 replications is summarized in Table 3. The partially linear FLEXI–Haz model achieved the strongest overall predictive performance, with the fully nonparametric FLEXI–Haz model generally ranking second. Zhong et al. (2022) model and Random Survival Forest performed less accurately overall, while Cox PH showed the weakest survival-probability prediction performance. The direct comparison with the known true survival functions shows the same general pattern, with the partially linear FLEXI–Haz model producing the smallest survival-prediction errors.

The concordance index shows a somewhat different ordering, with Zhong et al. (2022) model and Random Survival Forest achieving higher values. This does not contradict the other prediction

Model	Parameter	Estimate	Bias	Emp. SD	Est. SE	Cov. 90%	Cov. 95%
FLEXI–Haz	θ_1	1.993	-0.007	0.037	0.035	0.865	0.945
	θ_2	-0.993	0.007	0.029	0.029	0.895	0.945
Zhong et al. (2022)	θ_1	1.790	-0.210	0.032	0.033	0.000	0.000
	θ_2	-0.900	0.100	0.027	0.029	0.035	0.055

Table 2: *Estimation and inference for the linear coefficients across 200 simulation replications. Emp. SD denotes the empirical standard deviation of the estimates, and Est. SE denotes the mean estimated standard error.*

results, because the concordance index measures risk ranking rather than the accuracy or calibration of the estimated survival probabilities over the full follow-up period.

Model	IBS	Mean Brier	Mean AUC	C-index	Mean absolute bias	RMSE
FLEXI–Haz (partial linear)	**0.1139 (0.0005)**	**0.1125 (0.0005)**	**0.8677 (0.0004)**	0.7368 (0.0035)	**0.0183 (0.0016)**	**0.0315 (0.0022)**
FLEXI–Haz (nonlinear)	**0.1152 (0.0006)**	**0.1138 (0.0006)**	**0.8654 (0.0008)**	0.7438 (0.0043)	**0.0290 (0.0024)**	**0.0474 (0.0035)**
Zhong et al. (2022)	0.1184 (0.0006)	0.1171 (0.0006)	0.8561 (0.0006)	**0.7757 (0.0006)**	0.0488 (0.0014)	0.0767 (0.0016)
Random Survival Forest	0.1192 (0.0006)	0.1178 (0.0006)	0.8583 (0.0007)	**0.7709 (0.0015)**	0.0518 (0.0010)	0.0781 (0.0013)
Cox PH	0.1360 (0.0006)	0.1344 (0.0006)	0.8363 (0.0002)	0.7539 (0.0001)	0.0964 (0.0010)	0.1441 (0.0002)

Table 3: *Prediction performance across 200 simulation replications. Values are reported as mean (standard deviation). Lower values are better for IBS, Brier score, absolute bias, and RMSE, while higher values are better for AUC and concordance index. The two best values in each metric are shown in bold.*

Figures 1–3 show how predictive performance evolved over follow-up time. The differences between the methods become more pronounced as time increases. Both FLEXI–Haz models maintain higher time-dependent AUC and lower prediction error over most of the follow-up period, with the partially linear model generally providing the strongest performance. The advantage is particularly clear for survival RMSE, where the partially linear FLEXI–Haz curve remains substantially below those of the proportional-hazards comparators and Random Survival Forest at later times.

The Brier-score curves show a similar pattern. The methods perform comparably near the beginning of follow-up, when most subjects remain event-free, but the Cox model gradually separates from the other methods as the effects of the nonlinear and time-dependent components accumulate. FLEXI–Haz maintains the lowest or nearly lowest Brier score throughout the main part of the follow-up period. Together with the coefficient results in Table 2, these findings show that the partially linear FLEXI–Haz formulation benefits from using the known structural separation between the linear and flexible covariate effects, while retaining the flexibility required to model non-proportional hazards.

5.2 Rotterdam Breast Cancer Study

Table 4 summarizes the estimated ER-specific treatment effects for partially linear FLEXI–Haz, Zhong et al. (2022) model, and the full-data Cox model.

The partially linear FLEXI–Haz model indicates a stronger protective association of hormonal therapy in the high-ER group than in the low-ER group. In particular, the high-ER hormonal effect is clearly negative, while the corresponding low-ER confidence interval includes zero. FLEXI–Haz also estimates negative chemotherapy associations in both ER groups, with both 95% confidence intervals lying below zero. In contrast, the Zhong et al. (2022) model produces weaker treatment associations, with all four averaged 95% confidence intervals including zero. The full-data Cox analysis identifies a negative hormonal-therapy association in the high-ER group, while its remaining treatment-effect confidence intervals include zero. Overall, FLEXI–Haz yields stronger estimated

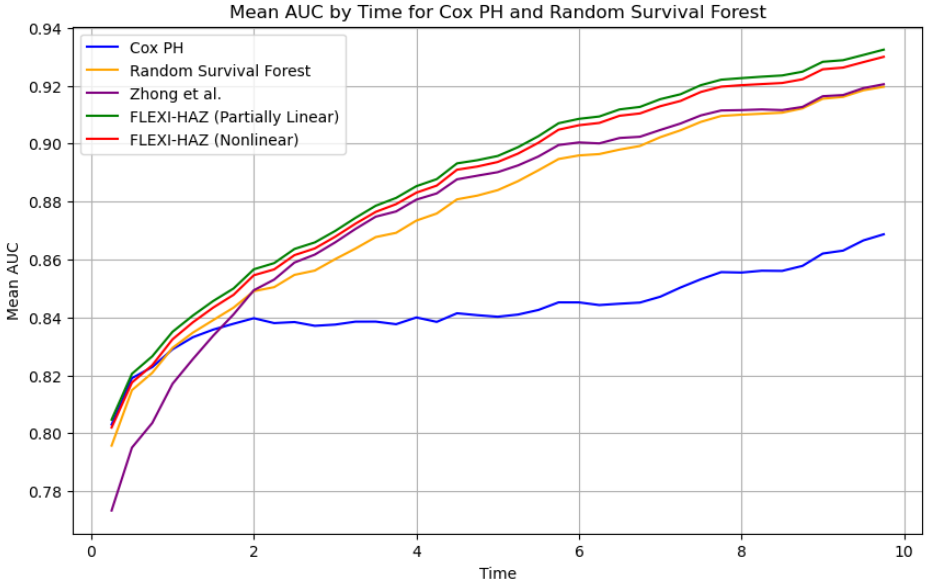


Figure 1: *Mean time-dependent AUC across the 200 simulation replications as a function of follow-up time (higher is better).*

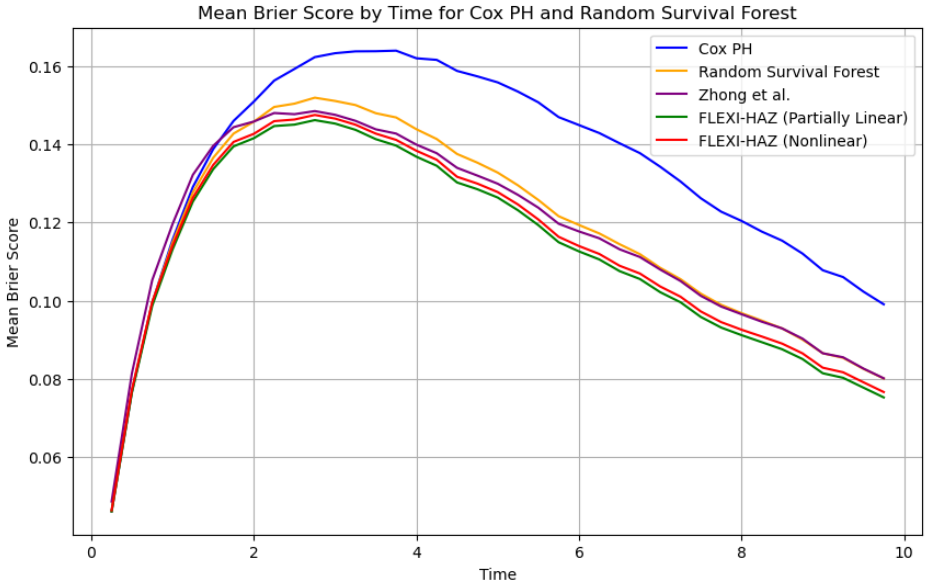


Figure 2: *Mean Brier score across the 200 simulation replications as a function of follow-up time (higher is better).*

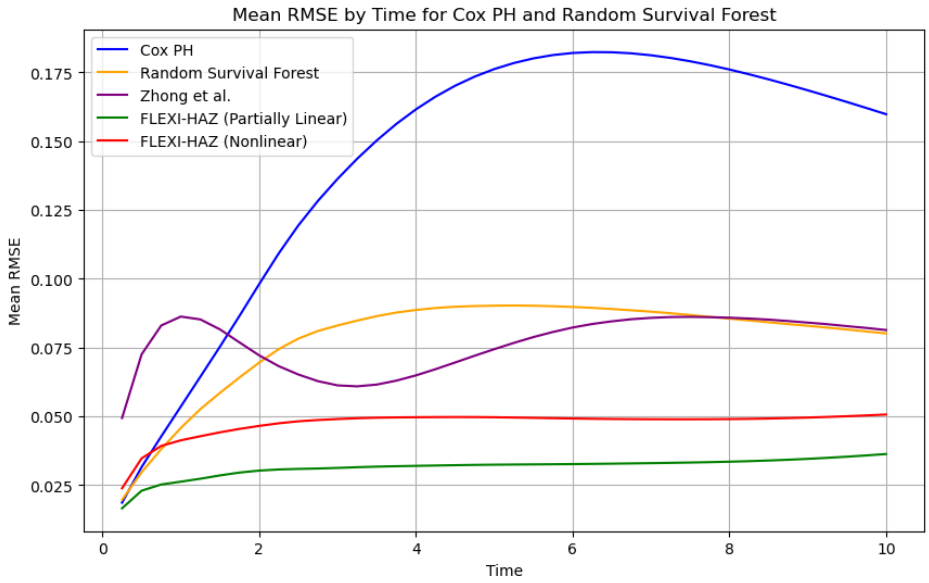


Figure 3: *Mean RMSE between the estimated and true survival probabilities across the 200 simulation replications as a function of follow-up time (higher is better).*

ER-specific treatment associations than the proportional-hazards alternatives, while allowing the effects of the remaining covariates to vary flexibly with time.

Treatment	ER	FLEXI–Haz	Zhong et al. (2022)	Cox PH
Hormonal	Low	−0.190 (−0.493, 0.113)	0.093 (−0.182, 0.368)	0.083 (−0.123, 0.289)
Hormonal	High	**−0.496 (−0.784, −0.207)**	−0.159 (−0.411, 0.093)	**−0.306 (−0.508, −0.104)**
Chemo	Low	**−0.309 (−0.590, −0.028)**	−0.029 (−0.210, 0.153)	−0.074 (−0.235, 0.086)
Chemo	High	**−0.352 (−0.638, −0.066)**	−0.132 (−0.355, 0.092)	−0.098 (−0.289, 0.093)

Table 4: *Estimated ER-specific treatment effects in the Rotterdam breast cancer study. Entries are log-hazard coefficient estimates with 95% confidence intervals in parentheses. Negative coefficients indicate lower estimated hazard. Statistically significant estimates, whose 95% confidence intervals exclude zero, are shown in bold.*

Predictive performance is summarized in Table 5 and Figures 4 and 5. Across the aggregate metrics and over follow-up time, both FLEXI–Haz models generally outperform the Zhong et al. (2022) proportional-hazards model and The Cox PH model. The partially linear FLEXI–Haz model consistently achieves lower prediction error and stronger discrimination, while the fully nonparametric FLEXI–Haz model also performs competitively. Random Survival Forest attains the strongest values for several metrics, but its performance is close to that of the partially linear FLEXI–Haz model, both in the aggregate results and across most of the follow-up period.

Taken together, these results highlight the main practical advantage of the partially linear FLEXI–Haz formulation. It achieves predictive performance that is comparable to the flexible Random Survival Forest and stronger than the neural proportional-hazards comparator, while simultaneously retaining an interpretable parametric component for the ER-specific treatment effects. In contrast to Random Survival Forest, FLEXI–Haz therefore provides not only flexible survival prediction but also coefficient estimates, standard errors, and confidence intervals for the covariates of primary interest.

Model	IBS	Mean Brier	Mean AUC	C-index
FLEXI–Haz (partial linear)	**0.1959 (0.0103)**	**0.1956 (0.0104)**	**0.7258 (0.0227)**	**0.6574 (0.0140)**
FLEXI–Haz (nonlinear)	0.2013 (0.0132)	0.2011 (0.0138)	0.7151 (0.0153)	0.6559 (0.0073)
Zhong et al. (2022)	0.2141 (0.0182)	0.2138 (0.0186)	0.6122 (0.1026)	0.5825 (0.0757)
Random Survival Forest	**0.1922 (0.0091)**	**0.1919 (0.0093)**	**0.7308 (0.0194)**	**0.6604 (0.0057)**
Cox PH	0.1861	0.1854	0.7181	0.6680

Table 5: *Prediction performance in the Rotterdam breast cancer study. For the four cross-validated models, values are reported as mean (standard deviation) across the five outer folds. The two best out-of-sample values in each metric are shown in bold.*

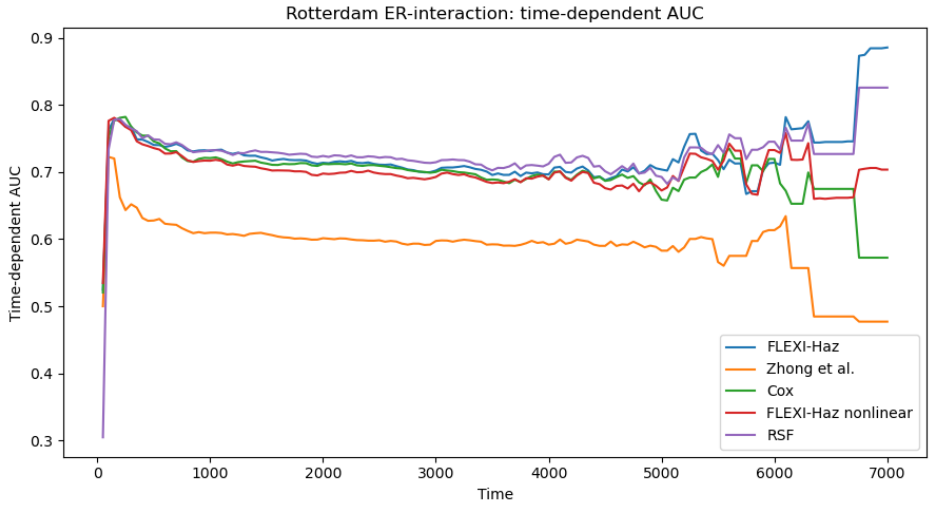


Figure 4: *Mean time-dependent AUC in the Rotterdam breast cancer study as a function of follow-up time.*

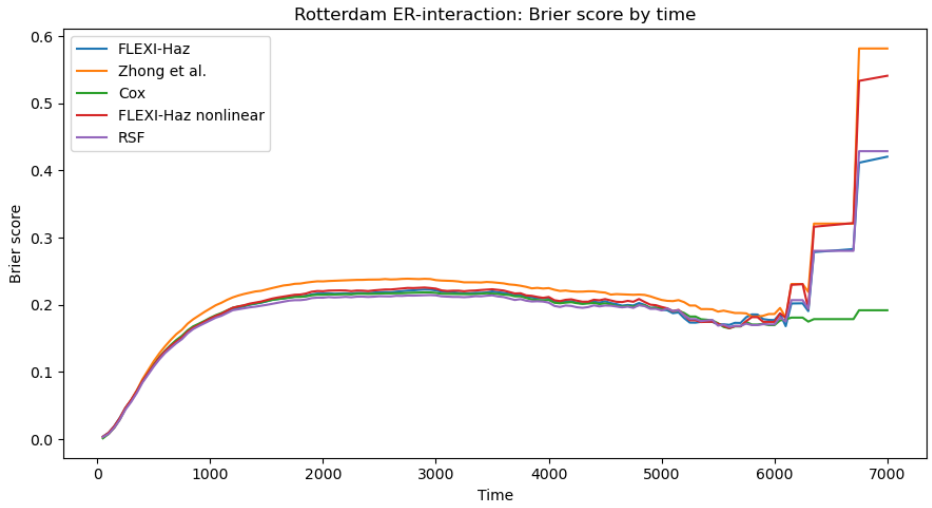


Figure 5: *Mean Brier score in the Rotterdam breast cancer study as a function of follow-up time.*

References

Ben Arie, A. and M. Gorfine (2025). Fflexible deep neural networks for partially linear survival data: Estimation and survival inference. *arXiv preprint arXiv:2512.10570*.

Cox, D. R. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological) 34*(2), 187–202.

Ishwaran, H., U. B. Kogalur, E. H. Blackstone, and M. S. Lauer (2008). Random survival forests. *The Annals of Applied Statistics 2*(3), 841 – 860.

Zhong, Q., J. Mueller, and J.-L. Wang (2022). Deep learning for the partially linear cox model. *The Annals of Statistics 50*(3), 1348–1375.